

AIRFLOW ENVIRONMENTAL SOLUTIONS

AES Operations & Finance Management System

Detailed System Design & Process Flow Document

Version 1.0 | July 2026 | Status: Draft for Developer Engagement

Technology stack: NestJS (backend) · Flutter (mobile & web) · PostgreSQL · Microsoft 365 / Fabric (BI)

CONFIDENTIAL — Internal and appointed development partners only

Table of Contents

1. Purpose and Vision

Airflow Environmental Solutions (AES) currently runs its operational cashflow, tax tracking, payroll and timesheets on a set of interlinked Excel workbooks. These have served well but are reaching their limits: concurrent multi-user entry, approval workflows, site-level data capture, audit trails and real-time visibility of business health cannot be delivered reliably in spreadsheets.

This document specifies a custom, full-stack business management system to be built by an engaged development team. It consolidates everything discussed and analysed to date: the existing **Operational Cashflow Report** workbook and its user manual, site payrolls (Mimosa, Unki, Head Office), the AES manhours registers and daily timesheets, plus the new requirements for petty cash, cash requisitions, travel allowances, budget authorisation, director withdrawals, access control and business-danger alerting.

1.1 Primary business objective

The owner's core requirement, in their own framing: *“see the core of the business — how money is coming in and going out, always see debt and interest, see incoming orders versus salaries and expenses, know whether the business is in danger, and be alerted to imminent danger.”* Every module in this design ultimately feeds that single view: a live Business Health Command Centre with automated danger alerts.

1.2 Design principles

- **Single source of truth.** One relational database (PostgreSQL) replaces all linked workbooks. Excel remains an output format (reports, bank schedules, client manhours), never the live data store.
- **Approve first, fund second.** Approval decisions are separated from fund availability. A request can be fully approved yet flagged “Pending Funds”, with its deadline driving escalation.
- **Human executes money movement.** The system prepares, approves, posts and instructs — but an authorised person always performs the actual bank/wallet transfer and captures the reference. This is deliberate internal control, not a technical gap.
- **Multi-currency native.** USD and ZWG (ZiG) are first-class citizens. Every money field stores currency, amount, rate used and rate type (official / parallel / client ratio). Nothing is converted destructively.
- **Site-first capture.** Each site (Mimosa, Unki, Zimplats, Head Office and future sites) captures its own timesheets and requests, with offline tolerance for poor mine connectivity.
- **Auditability everywhere.** Every insert, update, approval and status change is written to an immutable audit log with actor, timestamp and before/after values.

2. Scope and Module Map

The system is delivered in phases. Phase 1 is the financial core; Phase 2 adds payroll and timesheets; Phase 3 adds business development (CRM). All phases share one database, one authentication layer and one approval engine.

Phase	Module	Summary
1	Clients & Contracts	Master data; contract budgets, claims-to-date, over/under-claim variance (as per current workbook)
1	Orders & Order Expenses	Ad-hoc orders (Zimplats, Mimosa, Unki), VAT, health status, profitability, loan interest allocation
1	General Expenses & Overheads	Non-order costs; monthly overheads incl. salaries summary, PAYE tracking
1	Loans & Interest	Working-capital loans linked to orders, weekly interest, repayment tracking
1	Tax Ledger & ZIMRA	Output/input VAT, PAYE, brought-forward debt, ZIMRA interest, tax dashboard
1	Petty Cash	Floats per site, withdrawals, currency conversions, FD approval above threshold
1	Cash Requisitions	Online request → approval → fundability check → disbursement, with required-by deadlines
1	Travel Allowances	Per-diem requests, approval, advance and retirement (acquittal) of the advance
1	Budgets	Finance inputs budgets; dual authorisation by Operations Director and Finance Director
1	Director Withdrawals	Request → co-approval → posted to accounts ledger → manual transfer method → completion
1	Business Health & Alerts	Command centre: cash position, debt, interest, orders vs salaries+expenses, danger scoring, alerting
1	Admin Control Panel	Users, roles, thresholds, exchange rates, approval chains, audit log viewer
2	Timesheets	Per-site daily capture: Normal, OT 1.5, OT 2.0, Underground, Night; monthly summaries; client manhours export
2	Payroll	Multi-currency site payrolls with per-client USD/ZWG ratios; fixed head-office/director pay; statutory returns; bank schedules; payslips; Sage journal export
2	HR-lite	Employee master, leave days, grades, NEC classifications (needed by payroll; flagged as an oversight — see §16)
3	Business Development (CRM)	Contacts, interactions, pipeline, conversion of contacts → opportunities → orders/contracts

3. System Architecture

3.1 Chosen stack

Layer	Technology	Notes
Mobile app	Flutter (iOS + Android)	Site timesheet capture, requisitions, approvals on the go, dashboards, push notifications
Web front end	Flutter Web	Same codebase as mobile; full admin panel and finance workstation run here
API backend	NestJS (Node.js, TypeScript)	REST + WebSocket; modular architecture mirrors the module map; class-validator DTOs; OpenAPI docs
Database	PostgreSQL 16	Free, open source; strong relational integrity; row-level security; TimescaleDB optional for rate history
ORM / migrations	Prisma or TypeORM	Developer choice; migrations version-controlled
Auth & identity	Microsoft Entra ID (Azure AD) via OIDC	Staff already have M365 accounts; single sign-on; roles mapped to Entra groups or app-level roles
Jobs & queues	BullMQ (Redis)	Scheduled danger scans, interest accrual, deadline escalations, report generation
File storage	SharePoint/OneDrive via Microsoft Graph, or S3-compatible store	Receipts, fiscal tax invoices, payment vouchers, payslip PDFs
BI & analytics	Microsoft Fabric (Lakehouse + Power BI)	Nightly/near-real-time sync from PostgreSQL; executive dashboards; heavy analytics kept out of the OLTP database
Notifications	Push (FCM), email (Graph/Outlook), Teams messages	All alerts fan out to the channels each user selects
Reporting/exports	Server-side generation of XLSX, PDF, CSV	Bank schedules, payslips, manhours registers, ZIMRA-format reports, Sage journal CSV

3.2 Why PostgreSQL (answering the free-DBMS question)

Earlier options considered were Dataverse and PostgreSQL. Since the decision is now a fully custom NestJS/Flutter build, **PostgreSQL is the correct choice**: it is free and open source, integrates natively with NestJS ORMs, supports the relational integrity this schema needs (orders → expenses → loans → payroll all foreign-keyed), and Microsoft Fabric can mirror or ingest from it for BI. Dataverse only makes sense inside a Power Apps build, which is no longer the direction. Best practice for the deployment: managed PostgreSQL (e.g. Azure Database for PostgreSQL to stay in the Microsoft estate) or a self-hosted instance with automated daily backups, point-in-time recovery enabled, and a read replica for reporting if Fabric sync load grows.

3.3 High-level data flow

1. Site users and head office capture data (timesheets, orders, expenses, requests) via Flutter mobile/web → NestJS API → PostgreSQL.
2. The approval engine routes items to approvers; decisions post back and trigger notifications.
3. Scheduled jobs accrue loan and ZIMRA interest, recalculate order health, run danger scoring, and escalate deadline-bound items.

4. Approved financial events post to the internal Accounts Ledger (double-entry style journal), which is the base for cash position and health metrics.
5. PostgreSQL syncs to Microsoft Fabric; Power BI serves executive dashboards; operational dashboards render natively in Flutter from the API.
6. Exports are generated on demand or on schedule: bank schedules, payslips, statutory returns, client manhours, Sage journal entries.

4. Users, Roles and Access Control

Role-based access control (RBAC) is enforced in the NestJS API (guards on every endpoint), reflected in the Flutter UI (screens/buttons hidden per role), and mirrored in PostgreSQL row-level security for defence in depth. Roles are assignable per user per site, so a Site Clerk at Mimosa cannot see Unki data unless also granted there.

Role	Key permissions
Site Clerk / Timekeeper	Capture own site's timesheets and petty cash vouchers; raise requisitions; view own submissions
Site Manager	Approve site timesheets (first line); raise/endorse site requisitions and travel requests; site dashboards
Operations Staff	Create orders, order expenses; raise requisitions and travel requests; view own items
Finance Officer	Full transaction entry; record receipts/payments; manage petty cash floats; run payroll drafts; mark transfers complete; generate reports
Finance Director (FD)	Approve petty cash above threshold; co-authorise budgets; approve payroll; approve requisitions per matrix; tax dashboard
Operations Director (OD)	Co-authorise budgets; approve operational requisitions per matrix; contract/order oversight
Managing Director / Directors	Director withdrawals (request + co-approve); full financial summary; danger alerts always on
System Administrator	User/role management, thresholds, approval chains, exchange rates, reference data, audit log viewer, stuck-workflow reset
Auditor (read-only)	Read-everything, write-nothing role for external accountants/tax practitioners

4.1 Admin Control Panel

A dedicated admin area (web) provides: user and role administration; site management; approval-matrix configuration (which role approves what, at which monetary band, in which order); threshold settings (petty cash FD threshold, danger-alert trigger levels); exchange-rate management with full history; statutory rate management (VAT %, ZIMRA interest %, NSSA/ZIMDEF/NEC/MIPF parameters, PAYE bands per currency); notification channel configuration; audit log search; and delegation rules (an approver can delegate to a named alternate for a date range — essential so approvals never stall when a director travels).

4.2 Segregation of duties (built-in rules)

- No user may approve their own request, at any level, ever — enforced by the approval engine.
- The person who records a payment/transfer completion cannot be the requester of that item.
- Payroll: preparer (Finance Officer) and approver (FD) must be different users; the run locks after approval.
- Threshold and approval-matrix changes are themselves audited and require System Administrator role; optionally dual-control (second admin confirms).

5. Multi-Currency Design (USD / ZWG)

Zimbabwe's multi-currency reality is modelled explicitly, based on how AES actually operates (verified against the uploaded payrolls):

- **Currencies:** USD and ZWG (ZiG). The schema supports adding others (ZAR) without change.
- **Exchange rates:** an effective-dated *exchange_rates* table stores official (RBZ) and parallel (street) rates. Rates are never overwritten — a new row supersedes — so every historical transaction can always reproduce the rate it used. The current workbook's single Settings cell becomes a full rate history (this fixes the manual's own warning that conversions “drift”).
- **Client pay ratios:** each client contract carries a *currency split ratio* (e.g. Mimosa March 2025 payroll shows per-employee USD/ZWL ratios such as 0.4697 for the Site Manager). Employees assigned to a client site inherit that client's ratio for the pay period; the ratio itself is captured per payroll run so later rate movements do not rewrite history.
- **Ratio-free pay:** head office staff and directors are flagged *fixed-split* — their USD/ZWG split (often 50/50, or bespoke per person as seen for the Operations Director) is set on the employee record, not derived from any client. The payroll engine supports both modes in one run.
- **Storage rule:** every monetary value is stored as (amount, currency, fx_rate_id used, rate_type). Reporting can then present any figure in USD-equivalent at official rate, street rate, or as-transacted — mirroring the current workbook's side-by-side official/street comparison.
- **Conversions:** petty cash and treasury conversions (USD↔ZWG) are first-class transactions recording both legs, the rate achieved, the rate type, and any variance versus official rate — giving directors visibility of conversion cost/gain (a specific monitoring requirement).

6. Data Model (Core Entities)

The schema below is the developer's blueprint. Names are indicative; the ORM layer will formalise types and constraints. All tables carry: id (UUID), created_by, created_at, updated_by, updated_at, and are covered by the audit trigger. FK = foreign key.

6.1 Master data

Table	Key fields	Notes
sites	name, type (mine site / head office), client_id FK nullable, active	Mimosa, Unki, Zimplats, Head Office...
clients	name, contacts, payment_terms, currency_ratio_default, vat_number	From current Clients sheet
employees	works_no, names, national_id, nssa_no, grade, nec_class, occupation, site_id FK, employment_type, pay_mode (client-ratio fixed-split), fixed_usd_pct, hourly_rate, bank, branch, account_no, account_currency, leave_balance, active	Consolidates fields observed across all three payroll files
users	entra_object_id, employee_id FK nullable, email, status	Auth identities; linked to employees where applicable
roles / user_site_roles	role, user_id FK, site_id FK	RBAC per site
exchange_rates	date_effective, currency_pair, official_rate, parallel_rate, source, entered_by	Append-only history
settings / statutory_rates	vat_pct, zimra_interest_pct, nssa params, zimdef_pct, nec/mipf params, paye_bands (per currency, effective-dated)	Replaces Settings sheet; all effective-dated
approval_matrix	module, min_amount, max_amount, currency, step_order, approver_role, mode (sequential parallel either)	Drives the generic approval engine

6.2 Revenue side

Table	Key fields	Notes
contracts	client_id FK, title, value_ex_vat, start, end, monthly_budget (derived), status	Long-running contracts (Unki refugee chambers, Mimosa roadway/ventilation)
contract_claims	contract_id FK, period, amount_ex_vat, vat, vat_paid_to_date, received_date	Current Contract Payments sheet; feeds over/under-claim variance
orders	order_no, title, client_id FK, issue_date, closing_date, amount_ex_vat, vat (derived), serviced_bool, serviced_date, advance_payment_bool, status/health (derived)	Heart of the workbook, preserved
order_receipts	order_id FK, date, amount, currency, fx_rate_id, rate_type	Money in; ZWG receipts keep official + street USD views
order_expenses	order_id FK, description, category, amount_ex_vat, vat_paid_bool, input_vat (derived), attachment_id	Feeds spent-to-date, profit, input VAT

6.3 Cost, treasury and tax side

Table	Key fields	Notes
general_expenses	date, description, category, amount_ex_vat, vat_paid bool, comment, attachment_id	
overheads	period, salaries, paye_due, paye_paid, software, internet, rentals, other	Later fed automatically by the payroll module
loans	lender, order_id FK nullable, principal_usd, weekly_rate_pct, start_date, status	Interest accrual job writes loan_interest rows weekly
loan_repayments / loan_interest	loan_id FK, date, amount / period, interest	Outstanding balance derived, never stored stale
tax_ledger_entries	type (output VAT input VAT PAYE ZIMRA interest penalty), source_table, source_id, period, amount, currency	Everything the current Tax sheet consolidates, but event-sourced
other_tax_debt	type (PAYE VAT), principal, due_date, paid_to_date	Brought-forward debt; ZIMRA interest accrues per statutory rate; PAYE and VAT rows kept separate (improves on the workbook's combined-principal simplification)
zimra_assessments	type, period, assessed_amount, due_date, status	Tax Summary Dashboard input: discrepancy vs books, days overdue, interest
accounts / ledger_entries	account (bank USD, bank ZWG, petty cash per site, mobile wallet...); journal-style debit/credit entries with source reference	The internal Accounts Ledger every approved money event posts to; basis of cash position

6.4 New operational modules

Table	Key fields	Notes
petty_cash_floats	site_id FK, custodian_user_id, currency, float_amount, balance (derived)	One float per site per currency
petty_cash_txns	float_id FK, date, type (withdrawal top-up conversion-out conversion-in), amount, currency, purpose, order_id FK nullable, receipt attachment, status, approved_by	Conversions link two txns (both legs) with rate achieved
cash_requisitions	requester, site_id, purpose, amount, currency, required_by_date, status, funded_flag, order_id FK nullable	Status lifecycle in §8
travel_allowances	requester, trip_purpose, destination, date_from, date_to, per_diem_rate, days, advance_amount, status, retirement_status, retired_amount, receipts	Includes acquittal/retirement of the advance
budgets	period, department/site, line items (child table), total, status, prepared_by (Finance), od_approval, fd_approval	Dual authorisation
director_withdrawals	director_user_id, amount, currency, account_id FK, reason, status, co_approver, posted_ledger_entry_id, transfer_method, transfer_ref, completed_by	Posted to ledger before transfer; method chosen manually
approvals	subject_table, subject_id, step, approver_user_id, decision, decided_at, comment	One engine, all modules
alerts	type, severity, subject ref, message, raised_at, acknowledged_by	Danger alerts and operational nudges

Table	Key fields	Notes
audit_log	actor, table, record_id, action, before, after, at	Append-only; admin-readable

6.5 Timesheets and payroll (Phase 2)

Table	Key fields	Notes
timesheet_periods	site_id, month, status (open site-approved locked)	Locking prevents edits after payroll uses the data
timesheet_entries	period_id FK, employee_id FK, date, hours_normal, hours_ot15, hours_ot20, ug_shift bool/hours, night_hours, remarks	Matches the observed categories: Normal, OT 1.5, OT 2.0, Underground, Night
payroll_runs	site_id, period, status (draft checked approved paid locked), fx_rate_id, client_ratio_snapshot, prepared_by, approved_by	Ratio and rate frozen per run
payroll_lines	run_id FK, employee_id FK, basic_usd, basic_zwg, cola, ug_allowance, night_allowance, other_allowances, gross, paye, aids_levy, nssa_ee/er, zimdef, nec, mipf, nyaradzo, other_deductions, net_usd, net_zwg	Field set derived from the three uploaded payrolls (incl. AIDS levy split per currency)
statutory_returns	run_id(s), type (PAYE NSSA ZIMDEF NEC MIPF Nyaradzo), period, totals, due_date, remitted bool, remitted_date, ref	Compliance calendar drives reminders
bank_schedules	run_id FK, bank, currency, generated file ref	Per bank per currency, as in current Bank Schedule sheets
payslips	line_id FK, pdf ref, delivered_via, delivered_at	Email/portal/mobile delivery

7. The Generic Approval Engine

Every approvable item (requisition, travel allowance, budget, petty cash above threshold, director withdrawal, payroll run, timesheet period) flows through one shared engine, configured by the approval matrix. This is the most important build decision in the system: one engine means consistent behaviour, one audit trail format, one notification pattern, and new approval types added by configuration rather than code.

7.1 Engine behaviour

1. On submission, the engine reads the approval matrix for (module, amount band, currency, site) and instantiates the approval chain (sequential steps and/or parallel co-approvals).
2. Each approver is notified (push + email + Teams). The item appears in their in-app Approvals inbox with full context and attachments.
3. Approve / Reject / Return-for-correction, each with an optional comment. Return sends it back to the requester editable; Reject closes it.
4. Self-approval is blocked; delegation rules substitute an alternate approver when active.
5. SLA timers: if an approval sits beyond a configured time (e.g. 24h), reminders fire; beyond a second threshold it escalates to the next role or to directors.
6. Every decision writes to the approvals table and the audit log; the subject record's status advances automatically.

8. Cash Requisitions and Travel Allowances — Process Flow

8.1 Status lifecycle (shared pattern)

Draft → Submitted → Approved (*or Rejected / Returned*) → **Funds check** → Approved-Ready-to-Pay *or* Approved-Pending-Funds → Disbursed → (Travel only) Retired/Acquitted → Closed.

8.2 Detailed flow

1. Requester (any site or head office, mobile or web) completes the form: purpose, amount, currency, required-by date, linked order (optional), supporting attachment. Travel adds destination, dates and auto-computed per-diem (rate table by grade/destination class, admin-configurable).
2. Approval engine routes per matrix — e.g. Site Manager endorsement → Operations Director for operational requests, or straight to FD for finance items; bands by amount.
3. **Approval is decided on merit only — fund availability does not block approval.** This is an explicit business rule: finance accepts the obligation to complete whether or not funds exist today.
4. On approval the engine snapshots the live cash position (relevant accounts from the ledger). Funds sufficient → status Approved-Ready-to-Pay and Finance is notified to disburse. Funds short → status Approved-Pending-Funds; requester and Finance both notified with the shortfall amount.
5. A daily job re-tests all Pending-Funds items against the ledger and against their required-by dates: as the deadline nears (configurable, e.g. 3 days out) it escalates to FD; a missed deadline raises a danger-severity alert on the command centre.
6. Disbursement: Finance pays (bank, mobile money or petty cash), selects the source account, captures the reference; the engine posts the ledger entry and closes the item (travel moves to Awaiting Retirement).
7. Travel retirement: on return, the traveller submits receipts and unspent cash; Finance reconciles (refund due / owed); item closes. Unretired advances older than N days trigger reminders and appear on the FD dashboard.

9. Petty Cash — Process Flow

Each site (and head office) holds a petty cash float per currency with a named custodian. The module gives directors exactly the monitoring asked for: every withdrawal, every conversion, and FD approval above a threshold.

1. **Float establishment/top-up:** Finance requests a top-up (itself an approvable item); on disbursement the ledger moves money from bank to that site's petty cash account.
2. **Withdrawal below threshold:** custodian records the voucher (purpose, amount, currency, order link, photographed receipt). Site Manager confirms. Posted immediately; balance updates.
3. **Withdrawal at/above the FD threshold** (admin-configurable per currency, e.g. USD 100): the voucher routes through the approval engine to the **Finance Director** before cash may leave the box. The mobile app makes this a one-tap approval with the receipt image inline.
4. **Currency conversions:** converting float USD $\leftrightarrow$ ZWG records both legs, the achieved rate, and computes variance versus the official rate of the day. Conversions above the threshold also require FD approval. A conversions report shows cumulative conversion gain/loss per site — the specific “monitor conversions” requirement.
5. **Reconciliation:** custodian performs a periodic (weekly/monthly, configurable) cash count in-app; variances beyond a tolerance raise an alert to FD and lock further withdrawals until resolved.
6. **Replenishment** is imprest-style: reimbursement equals vouchers since last top-up, keeping the float constant and self-auditing.

10. Budget Preparation and Dual Authorisation — Process Flow

Requirement: *finance team inputs budgets which are authorised by the Operations Director and Finance (Director).* Implemented as a dual-approval workflow with line-item detail:

1. Finance Officer creates a budget for a period and scope (site, department, contract or company-wide) with line items (category, description, amount, currency). Templates can be cloned from prior periods.
2. Submission triggers parallel co-approval: Operations Director and Finance Director both receive it. Both must approve for the budget to become Active; either can Return with comments, which restarts after correction.
3. Active budgets become the yardstick: actual spend (order expenses, general expenses, overheads, requisitions) is tracked against budget lines in real time, with percentage-consumed indicators and over-budget alerts at configurable levels (e.g. warn at 85%, alert at 100%).
4. Budget revisions are versioned — a revision is a new approvable object; history of every version and its approvals is preserved.
5. Operations-originated budget requests (e.g. a site needing an unbudgeted amount) enter as Budget Change Requests through the same dual-authorisation path — satisfying the “automate budget approval from operations to directors” requirement.

11. Director Withdrawals — Process Flow

As agreed: the system automates the request, approval and posting; the transfer method is then chosen and executed manually, keeping a human in control of actual money movement.

1. A director raises a withdrawal: amount, currency, destination account, reason.
2. Co-approval by a second director (configurable: specific co-signatory or any other director) via the approval engine. Self-approval impossible by design.
3. On approval, the system checks fund availability (informational, using the same approve-first pattern) and posts the withdrawal to the Accounts Ledger as “Posted — Awaiting Transfer”, so the financial summary and cash position reflect it immediately.
4. Finance (or the director, per policy) opens the posted item, selects the transfer method (EFT / RTGS / mobile money / cash), executes it in the banking channel manually, then captures the reference number and marks Completed.
5. A nudge job flags Posted items unactioned beyond a configurable age; the full trail (request → co-approval → posting → method → reference → completion) is visible to all directors and the auditor role.

12. Timesheets — Process Flow

Modelled directly on the current practice observed in the AES timesheet and manhours workbooks: daily hour capture per employee in five categories (Normal, Overtime 1.5, Overtime 2.0, Underground, Night), summarised monthly per site, and reported to clients as contractor manhours.

1. Each site's Timekeeper/Clerk captures daily hours per employee on mobile (grid view: employees down, days across — same mental model as the current sheets). Offline capture is supported; entries sync when connectivity returns (mines have dead zones).
2. Validation at entry: max hours/day, category exclusivity rules (configurable), automatic flag of anomalies (e.g. hours on a leave day — links to HR-lite leave records).
3. Site Manager reviews and approves the site's period (first-line approval). The period then locks for edits; corrections require a reopening request (audited).
4. Head office captures its own timesheet the same way (fixed 208-hour standard observed in payrolls prefills; exceptions edited).
5. Approved timesheet summaries flow straight into the payroll engine — hours × rate, UG shifts, night hours and OT premia computed without retyping (removing today's manual transfer between the timesheet workbook and paysheets).
6. Client manhours registers (the format in the AES Manhours workbook, including SHE-relevant totals) are generated as XLSX/PDF exports per site per month for client submission.

13. Payroll — Process Flow

The payroll engine reproduces, then improves, the three observed payroll patterns: Mimosa (client-ratio, hourly, shift allowances, full statutory stack incl. NEC/MIPF), Unki (similar) and Head Office (fixed salaries, simpler statutory set, Sage journal).

13.1 Run lifecycle

1. Finance Officer opens a run for (site, period). The engine pulls: approved timesheet hours, employee master (rates, grades, pay mode, banks), the period's exchange rate (chosen and frozen into the run), and the client ratio snapshot for client-ratio employees.
2. Gross build-up per employee: basic (hours × rate, or fixed salary), overtime at 1.5×/2.0×, underground shift allowance, night allowance, COLA, other allowances — all split USD/ZWG per the applicable ratio or fixed split.
3. Statutory calculations per currency: PAYE per current ZIMRA bands with AIDS levy (3% of PAYE), computed separately for the USD and ZWG portions exactly as the current sheets do; NSSA (employee+employer within the insurable ceiling); ZIMDEF (employer); NEC and MIPF where applicable (mining sites); Nyaradzo and other voluntary deductions.
4. Draft paysheet review → FD approval through the approval engine → run locks.
5. Outputs generated automatically: bank schedules per bank per currency (FBC Zvishavane pattern observed), payslips (PDF, delivered by email/portal/app), statutory return summaries (PAYE, NSSA, ZIMDEF, NEC, MIPF, Nyaradzo) with due dates onto the compliance calendar, and a Sage journal entry export matching the existing “Sage Journal Entry” sheet format.
6. Posting: the approved run posts salaries and employer costs to the Accounts Ledger and Overheads automatically — which is what powers the owner's “incoming orders vs salaries and expenses” view without any retyping.

13.2 Payroll rules of note

- Statutory parameters (PAYE bands per currency, NSSA ceiling, levies) are effective-dated admin data, not code — Zimbabwe changes these often.
- Per-employee overrides are supported (observed in the files: individual ratios differing within one site, bespoke director splits).
- A payroll cannot be run on an unapproved timesheet period; an approved run cannot be edited, only reversed by a correcting run (both preserved).

14. Business Health Command Centre and Danger Alerts

This is the module the whole system exists to feed. It answers, at a glance and in real time: how is money coming in and going out; what do we owe and what interest is it accruing; how do incoming orders compare to salaries and expenses; are we safe, at risk, or in imminent danger.

14.1 The command centre (web + mobile)

Panel	Content
Cash position	Live balances per account (bank USD, bank ZWG, petty cash per site, wallets), total USD-equivalent at official and street rates, 30/60/90-day cash trend
Money in vs out	Rolling inflows (order receipts, contract claims) vs outflows (expenses, overheads, payroll, tax, loan service) — daily/weekly/monthly views
Debt & interest watch	Every loan: principal, accrued interest, weekly interest burn rate; ZIMRA debt with accruing interest; total debt service due in the next 30/60/90 days — the owner's "always see my debt and interests" view
Orders vs payroll & expenses	Pipeline of open + serviced-unpaid orders (expected in) charted against committed salaries, overheads and approved requisitions (expected out) for the coming period — the coverage ratio is a headline number
Receivables ageing	Serviced-awaiting-payment orders bucketed 0-30/31-60/61-90/90+, overdue highlighted, per client
Tax exposure	Net VAT and PAYE payable, ZIMRA assessments, days overdue, interest accruing — from the tax ledger
Pending obligations	Approved-Pending-Funds items with deadlines; unfunded gap total
Health verdict	The single HEALTHY / WATCH / ACT indicator (carried over from the current Financial Summary), now computed continuously with drill-down to what is driving it

14.2 Danger scoring and alerting

A scheduled engine (hourly for cash, daily for the rest) evaluates configurable rules; each rule has a severity and channels. Directors always receive Danger severity. Initial rule set:

Rule (admin-tunable)	Default trigger	Severity
Cash runway	Projected cash covers < X weeks of committed outflows (payroll + overheads + debt service)	Danger at <4 weeks; Watch at <8
Payroll coverage	Next payroll total exceeds available liquid cash	Danger
Coverage ratio	Expected receipts (next 30d) / committed outflows (next 30d) below threshold	Watch <1.2; Danger <1.0
Overdue receivables spike	90+ bucket exceeds % of total receivables	Watch
Loan interest burn	Weekly interest exceeds % of average weekly receipts	Watch/Danger bands
ZIMRA overdue	Any assessed amount past due (interest + penalty risk up to 100%)	Danger
Deadline breach	Approved-Pending-Funds item misses its required-by date	Danger
Petty cash variance	Reconciliation variance beyond tolerance	Watch

Rule (admin-tunable)	Default trigger	Severity
Concentration risk	One client > X% of open order value	Watch
Conversion loss	Cumulative conversion loss vs official rate beyond threshold in period	Watch

Alerts fan out by severity: in-app always; push and email for Watch+; for Danger, push + email + Teams to all directors, repeated until acknowledged in-app. Every alert links straight to the drill-down that explains it. Acknowledgement and resolution are audited.

15. Mobile App and Reporting

15.1 Flutter mobile app — screen map

Area	Screens / capabilities	Primary users
Home	Role-aware dashboard tiles; danger banner if active	All
Approvals inbox	One-tap approve/reject with context, attachments, comments; biometric confirm for money items	Managers, Directors, FD, OD
Timesheets	Daily grid capture; offline-first with sync; period submit	Site Clerks, Site Managers
Requests	Raise/track requisitions, travel allowances, petty cash vouchers; camera receipt capture	All staff
Petty cash	Custodian float view, vouchers, conversions, reconciliation count	Custodians, Finance
Orders & receivables	Order health board, receivables ageing, mark serviced	Ops, Finance
Command centre	Health verdict, cash, debt & interest, orders-vs-payroll coverage, alert feed	Directors, FD, OD
Director actions	Raise withdrawal; co-approve; view posted-awaiting-transfer	Directors
Notifications	Push via FCM; alert acknowledgement	All

Flutter Web reuses the same codebase for the full workstation experience (finance processing, payroll runs, admin panel, big-screen dashboards). Offline strategy: timesheets and draft requests persist locally (SQLite/Drift) and sync with conflict detection; everything else requires connectivity.

15.2 Reporting and Microsoft Fabric

- **Operational reports (from the API, on demand/schedule):** all current workbook outputs are preserved as system reports — Orders Dashboard, Receivables, VAT obligations, Tax ledger and Tax Summary, Performance, Financial Summary — plus bank schedules, payslips, statutory returns, client manhours, budget-vs-actual, petty cash and conversions reports, and the Sage journal export. Formats: XLSX, PDF, CSV.
- **Fabric layer:** PostgreSQL data lands in a Fabric Lakehouse (mirroring or scheduled pipeline). Power BI models serve executive analytics — trends across years, site profitability, client profitability, payroll cost evolution, conversion-loss analysis — without loading the transactional database. Report distribution through the M365 estate the team already uses (Teams tabs, scheduled email subscriptions).
- **Design rule:** operational, second-by-second numbers come from the API; analytical, historical numbers come from Fabric. Both read the same source of truth.

15.3 Conversational analytics — talking to the data with AI in Fabric

A stated reason for choosing Fabric is the ability to interrogate the business data conversationally with AI. This is a first-class design goal, not a by-product, and it shapes how the Fabric layer must be built:

- **Capabilities targeted:** Copilot in Power BI/Fabric for natural-language questions over the semantic model (“which client has the worst receivables ageing?”, “what did conversion losses cost us at Mimosa last quarter?”, “how many weeks of payroll does current cash cover?”); and a Fabric Data Agent grounded on the Lakehouse, configured with AES business context, surfaced in Teams so directors can chat with the business from their phones.

- **Semantic model as the contract:** AI answers come from the semantic model, not the raw tables, so the model is a formal deliverable: business-friendly names, descriptions and synonyms for every table, column and measure (street rate, coverage ratio, order health, runway); all key metrics defined once as governed measures matching Appendix A's rules exactly. A weak model produces confident wrong answers about the finances — model quality is therefore an acceptance criterion, tested with a bank of ≥ 50 real business questions and expected answers.
- **Security carries into AI:** row-level and object-level security in the semantic model govern Copilot and agent responses — a site clerk asking about director pay must get nothing. The RBAC matrix (§4) is replicated as RLS/OLS roles in Fabric, and payroll tables are excluded from general-audience models entirely (a separate FD/Director-only model covers them). Sensitivity labels applied throughout.
- **Grounding and freshness:** the agent answers from the Lakehouse, so its knowledge is as fresh as the sync (§15.2). Time-critical questions (“can we cover payroll today?”) are answered authoritatively by the Command Centre from the API; the AI layer is for analysis and exploration. The agent's system context states its data freshness so answers are honestly framed.
- **Licensing check:** Copilot and Data Agents require paid Fabric capacity, and Microsoft has changed minimum SKU requirements over time — verify the current capacity requirement and cost against the tenant before finalising the architecture, and size the capacity for AI workloads, not just report refreshes.
- **Complementary option:** because all operational data lives in PostgreSQL behind the NestJS API, an in-app AI assistant (LLM over the API with the same RBAC) can later be added for transactional questions inside the Flutter app — independent of Fabric licensing. Phase 3 candidate.

16. Identified Oversights and Recommendations

Areas reviewed on request that were not (fully) in the stated requirements but which the uploaded files and the design analysis show are needed:

#	Oversight	Recommendation
1	Employee master / HR-lite is a prerequisite for payroll and timesheets, but was not requested	Include HR-lite in Phase 2: employee records, leave balances (leave days appear in Head Office payroll), grades/NEC classes, bank details, start/end dates
2	Sage integration exists today (Sage Journal Entry sheets) but was not mentioned	Keep it: generate Sage-format journal exports per payroll run and (optionally) per period for the general ledger; confirm the exact Sage product/version with the accountant
3	Statutory compliance calendar	PAYE, VAT, NSSA, ZIMDEF, NEC, MIPF have fixed remittance deadlines; missing them triggers ZIMRA/NSSA interest and penalties. Build a compliance calendar with reminders and remittance capture — it also feeds the danger engine
4	Corporate income tax still unmodelled	The workbook explicitly excludes income tax (25% + 3% AIDS levy). Add at least a provision estimate line to the health view so the verdict is not flattered; full computation stays with the tax practitioner
5	Attachment & fiscal invoice discipline	Input VAT is only claimable with valid fiscal tax invoices; make the invoice attachment mandatory (configurable) on VAT-claimed expenses
6	Travel advance retirement	Requested “travel allowances” only; without acquittal, advances leak. Retirement workflow included in §8
7	Approval delegation & availability	Directors travel; without delegation, workflows stall. Delegation rules included in §4.1
8	Offline capture at mine sites	Connectivity underground/at sites is unreliable; offline-first timesheets and drafts included in §15
9	Exchange-rate history	The single-cell rate in Settings loses history and distorts old records; effective-dated rate table included in §5
10	SHE reporting	A SHE Report sheet exists in the timesheet workbook; Phase 2 can add basic SHE incident/statistics capture per site alongside timesheets (client manhours reports often require it)
11	Data migration	A one-time, scripted migration from the current workbooks (clients, contracts, orders, expenses, loans, tax balances, employees) with a reconciliation report signed off by Finance before go-live
12	Backups & disaster recovery	Daily automated PostgreSQL backups, point-in-time recovery, restore test each quarter; file-store versioning
13	Payroll data privacy	Salaries, IDs and bank numbers are sensitive; encrypt at rest, restrict payroll tables to Finance/FD/Admin roles, mask bank accounts in UI except final digits, and audit every payroll view/export
14	Loan agreements repository	Loans are tracked numerically today; attach the actual agreements and repayment schedules to loan records
15	Multi-bank future	Only FBC observed; schema already supports multiple banks/wallets per employee and per company account

17. Future Version — Business Development Module (Phase 3)

To track business development contacts and their conversion into opportunities, as requested for a future version, Phase 3 adds a lightweight CRM sharing the same platform:

- **Contacts & organisations:** prospects and existing clients (linked to the clients table), owners (e.g. the Business Development Officer already on payroll), source of contact.
- **Interactions:** calls, visits, emails, tenders submitted — logged with dates and outcomes.
- **Pipeline:** stages (Contact → Qualified → Proposal/Tender → Negotiation → Won/Lost) with value estimates; a Won opportunity converts into an Order or Contract in one action, closing the loop with the finance core.
- **Conversion analytics:** contacts made vs opportunities created vs value won, per officer and per period — in Fabric, answering “how much converts into opportunities” directly.

18. Non-Functional Requirements and Delivery Plan

18.1 Non-functional requirements

Area	Requirement
Security	OIDC via Entra ID; MFA enforced for FD/Directors/Admin; TLS everywhere; secrets in a vault; OWASP ASVS L2 as the bar; biometric confirm on mobile for approvals of money items
Auditability	Append-only audit log; payroll views/exports audited; approval trails immutable
Availability	Target 99.5%; graceful offline for site capture; status page for admins
Performance	Dashboard API responses < 2s for current-period views; payroll run for 60+ employees < 1 minute
Data protection	Encryption at rest and in transit; role-scoped access to payroll and personal data; Zimbabwe Data Protection Act (Cyber and Data Protection Act) compliance review before go-live
Backup/DR	Daily backups, PITR, quarterly restore drill, documented RPO 24h / RTO 8h (tighten later if needed)
Localisation	Currency formats, ZWG/USD presentation, Africa/Harare timezone, English UI
Extensibility	New sites, clients, currencies, statutory rates and approval rules are configuration, not code

18.2 Suggested delivery sequence for the development team

1. Foundations: auth (Entra ID), RBAC, audit, approval engine, exchange rates, admin panel skeleton, notification service.
2. Financial core: clients/contracts/orders/expenses/loans/tax ledger + Accounts Ledger + migration of existing workbook data.
3. Workflow modules: requisitions, travel, petty cash, budgets, director withdrawals.
4. Command centre + danger engine + alerts (the owner's primary deliverable — pull forward as early as data allows; a useful first version can run on the financial core alone).
5. Phase 2: timesheets → payroll → statutory outputs → Sage export → HR-lite.
6. Fabric/Power BI analytics layer; then Phase 3 CRM.
7. Each stage: UAT with Finance on real data, parallel-run against the Excel system for at least one full month before cut-over (payroll: two parallel cycles).

18.3 Acceptance criteria anchors (samples)

- A requisition approved with insufficient funds shows Approved-Pending-Funds, notifies both parties with the shortfall, and escalates 3 days before its required-by date.
- A petty cash withdrawal of USD 100+ cannot be posted without FD approval; below threshold it posts with Site Manager confirmation.
- A budget becomes Active only after both OD and FD approve; either Return restarts the cycle.
- A director withdrawal appears in the ledger and Financial Summary immediately on approval, before any transfer occurs; completion requires a transfer method and reference.
- The Mimosa March 2025 payroll, re-run in the system from the same inputs, reproduces the workbook's net pay per employee within rounding tolerance, in both currencies.
- With cash runway below four weeks of committed outflows, all directors receive a Danger alert within one hour and it persists until acknowledged.

This document is a management and engineering specification, not tax or legal advice. Statutory rates and rules referenced (VAT, PAYE bands, AIDS levy, NSSA, ZIMDEF, NEC, MIPF, ZIMRA interest) change frequently in Zimbabwe and must be verified with ZIMRA/NSSA and a registered tax practitioner at build time; the system therefore treats all of them as editable, effective-dated configuration.

Appendix A — Business Logic Specification (Extracted from the Current Workbook)

The rules below were extracted directly from the formulas in the *Operational Cashflow Report* workbook supplied with the manual. They are the authoritative statement of how AES currently computes each figure, and the new system must reproduce them (except where a deliberate improvement is marked **IMPROVE**). Developers should encode these as unit-tested domain services, not as report-layer arithmetic.

A.1 Order financials

Field	Rule (as implemented today)
VAT	$\text{order_value_ex_vat} \times \text{VATRate}$ (Settings, currently 15.5%)
Total incl VAT	$\text{order_value_ex_vat} + \text{VAT}$
Spent to date	Sum of Order Expenses (ex VAT) linked to the order number
ZiG receipts	Converted to USD at both the official and the street rate, shown side by side
Total received (official basis)	$\text{USD received directly} + (\text{ZiG received} \div \text{official rate})$
Total received (street basis)	$\text{USD received directly} + (\text{ZiG received} \div \text{street rate})$
Outstanding	$\text{Total incl VAT} - \text{total received (official basis)}$
Profit/loss ex VAT	$\text{order_value_ex_vat} - \text{spent_to_date}$
Loan interest charged to order	Sum of accrued interest of all loans linked to this order
Net profit after loan	$\text{profit_ex_vat} - \text{loan_interest}$
Margin %	$\text{profit_ex_vat} \div \text{order_value_ex_vat}$ (blank if value is zero)

IMPROVE: today's conversion uses the single current rate in Settings, so old ZiG receipts are re-valued whenever the rate changes. The new system stores the rate effective on the receipt date (\$5), preserving history; both official- and street-basis views remain available.

A.2 Order payment status and health (state machine)

Payment status: Paid when total received $\geq$ total incl VAT (with a 1-cent tolerance); Partially Paid when anything has been received; otherwise Unpaid.

Health is evaluated in strict order — the first matching rule wins:

Order	Condition	Health
1	Total received $\geq$ total incl VAT (tolerance \$0.01)	PAID (green)
2	Something received AND not yet serviced	ADVANCE PAID (blue)
3	Serviced AND today > closing date	OVERDUE PAYMENT (red)
4	Serviced AND not past closing date	AWAITING PAYMENT (amber)
5	Not serviced AND today > closing date	OVERDUE SERVICE (red)
6	Otherwise	OPEN (grey)

The new system re-evaluates health on every relevant event (receipt, service tick, date passing via the nightly job) and emits alerts on transitions into either red state.

A.3 Contract claim variance

- Contract months = whole months between start and end dates, inclusive (+1).
- Monthly budget = contract value ex VAT ÷ contract months.
- Invoiced to date = sum of that contract's claims (ex VAT) from Contract Payments.
- Months elapsed = months from start to today (+1), capped at contract months; zero before start.
- Should-have-invoiced = monthly budget × months elapsed.
- Variance = invoiced to date – should-have-invoiced. Positive (over-claimed) shows red; within budget shows green.
- Contract status: Upcoming before start; Completed after end; otherwise Active.

A.4 Loan interest

- **Accrual is simple (non-compounding) weekly interest:** interest = principal × weekly rate × weeks elapsed, where weeks elapsed = (today – start date) ÷ 7 as a fraction (not rounded).
- Total due = principal + accrued interest. Repaid = USD repayments + (ZiG repayments ÷ official rate). Balance = total due – repaid; status Settled when balance ≤ \$0.01.
- **IMPROVE:** interest accrues on the full principal regardless of repayments made — i.e. repayments do not reduce the interest base. The new system must confirm each lender's actual terms and support both methods (flat on principal, and reducing-balance), configurable per loan; the workbook's flat method is the default for continuity.
- **IMPROVE:** ZiG repayments convert at the current official rate, not the rate on the repayment date; fixed by effective-dated rates as in A.1.

A.5 Tax ledger consolidation

VAT (net payable) is built exactly as follows:

1. Output VAT = VAT on all serviced orders + VAT on all contract claims.
2. Input VAT = input VAT from Order Expenses + General Expenses + Overheads (where VAT-paid is flagged).
3. Net current VAT = output – input.
4. Add brought-forward VAT principal from Other Tax Debt.
5. Subtract VAT paid to date recorded on Orders and on Contract Payments.
6. Result = net VAT payable (can be negative, i.e. recoverable).

PAYE: PAYE due (Overheads) + brought-forward PAYE principal – PAYE paid (Overheads) = net PAYE payable.

In Phase 2 the payroll engine replaces the manual Overheads PAYE figures as the source.

Other Tax Debt interest = (PAYE principal + VAT principal) × ZIMRA rate × days overdue ÷ 365, days counted from the due date. **IMPROVE:** the new schema keeps PAYE and VAT debts as separate rows so their interest is tracked separately (the workbook combines them and its own manual flags this as a simplification).

A.6 ZIMRA reconciliation dashboard

- Discrepancy = payable per books – amount assessed by ZIMRA (entered), for VAT and PAYE separately.
- Days overdue = max(0, today – due date entered).

- $\text{ZIMRA interest} = \max(0, \text{payable per books}) \times \text{ZIMRA annual rate} \times \text{days overdue} \div 365$.
- **Total estimated tax liability** = $\max(0, \text{net VAT payable}) + \max(0, \text{net PAYE payable}) + \text{other-tax-debt outstanding balance} + \text{other-tax-debt accrued interest}$. Negative (recoverable) positions are floored at zero — a refund due does not offset the headline liability.

A.7 Performance (operating result)

- Total income = serviced order values (ex VAT) + contract claims invoiced (ex VAT). Unserviced orders are excluded from income.
- Total expenses = order expenses + general expenses + overhead totals + loan interest accrued.
- Operating profit = income – expenses; operating margin = profit ÷ income.
- Debt funding view: total loan principal taken, loan balance outstanding, and debt as % of total income.
- Pending obligations = $\max(0, \text{net VAT payable}) + \max(0, \text{total tax liability}) + \text{loan balance outstanding} + \text{other-tax-debt balance}$.

A.8 Financial Summary — health verdict thresholds

The one-glance verdict is computed from company-wide totals (all figures USD, official basis):

Verdict	Rule (first match wins)
ACT — obligations exceed cash received	outstanding receivables + total tax liability + loan balance > total cash received
WATCH — high receivables vs cash in	outstanding receivables > 50% of total cash received
HEALTHY — cash covers obligations	otherwise

Key ratios shown beside the verdict: overall margin (total profit ÷ total order value); receivables-to-cash-in; tax-liability-to-net-profit; loan-balance-to-cash-in.

IMPROVE: these lifetime-cumulative rules are preserved as the “classic verdict” for continuity, but the Command Centre (§14) supersedes them with time-windowed measures (cash runway, 30-day coverage ratio, payroll coverage), because cumulative totals become less sensitive as history grows — a business can be in acute danger while lifetime cash received still exceeds lifetime obligations. Both verdicts are displayed during the transition period.

A.9 Settings and named values

Named settings the formulas rely on — all become effective-dated admin configuration in the new system: VATRate (15.5%), OffRate (official RBZ USD/ZWG), StreetRate (parallel), ZimraInt (10% p.a.), default weekly loan rate, and RptDate (today; drives every ageing and interest computation — in the new system this is simply the server clock in Africa/Harare, with all interest accrued by the nightly job rather than recalculated volatily).

A.10 Acceptance test derived from this appendix

Migration acceptance: after importing the live workbook's data, the new system's computed values for every order's health, payment status, profit, loan interest, the tax ledger lines, the ZIMRA dashboard, Performance and the Financial Summary verdict must match the workbook's displayed values on the same report date,

within \$0.01 per line, using the workbook's classic rules. Only after that parity is signed off do the IMPROVE variants activate.

Appendix B — Requirements Traceability (Original Workbook Specification)

The workbook was originally commissioned against a detailed specification from the business owner. That specification is the requirements baseline for the finance core, and this appendix proves coverage: every requirement is mapped to the design section that satisfies it. Two items surfaced as gaps and are closed in B.2. Developers should keep this matrix updated as the build progresses — it doubles as the UAT checklist for the finance core.

B.1 Traceability matrix

#	Original requirement (summarised)	Where covered in this design
1	Orders page: client drop-down fed by a clients master page; order number, title, issue and closing dates, required amount; colour-coded health; spent-to-date from expenses; USD receipts; ZWL/ZiG receipts converted at current rate plus separate street-rate columns; serviced checkbox and date; advance-payment flag; profit/loss ex VAT; VAT at 15.5%	§6.1 clients, §6.2 orders / order_receipts / order_expenses; exact rules in Appendix A.1–A.2; dual official/street valuation in §5; VAT rate as effective-dated config (§6.1, A.9)
2	Orders summary dashboard: serviced vs not, overall health, charts and lists; pending, advance-paid, partially serviced and completed orders; profit per order after loan interest and expenses	Orders Dashboard preserved as a system report (§15.2); live order health board in mobile (§15.1); charts in Flutter + Power BI (§15.2); per-order net profit after loan in A.1. Partial servicing gap closed in B.2(a)
3	Serviced-awaiting-payment list (expected incoming cash) as a dashboard	Receivables report and receivables-ageing panel of the Command Centre (§14.1, §15.2)
4	Paid-orders VAT obligations dashboard; each VAT amount and VAT-paid-to-date flowing to the tax page	VAT_Paid report preserved (§15.2); event-sourced tax_ledger (§6.3); consolidation rules A.5
5	Contracts page: value, start/end dates, monthly invoiceable budget, over/under-claim visibility	§6.2 contracts; variance logic A.3
6	Monthly contract payments with description, contract drop-down, amount received, VAT received and VAT-paid-to-date flowing to tax	§6.2 contract_claims; A.5 steps 1 and 5
7	Monthly overheads: salaries, PAYE (due and paid, balance to tax), software, internet, rentals, other, comment	§6.3 overheads; PAYE flow A.5; superseded as source by payroll engine in Phase 2 (§13.1 step 6)
8	Loans page: amount per order, weekly interest rate, repaid to date, current amount due; USD with the same local-currency conversions	§6.3 loans / loan_repayments / loan_interest; exact interest math and IMPROVE variants A.4
9	Other tax debt: PAYE and VAT principals, due dates, interest formulas, paid-to-date; appears in tax dashboard	§6.3 other_tax_debt; interest rule A.5; ZIMRA dashboard A.6; kept as separate PAYE/VAT rows (improvement)
10	Tax page: VAT and PAYE in two tables extracted from all other pages	tax_ledger consolidation (§6.3, A.5); Tax report preserved (§15.2)
11	Order expenses table: drop-down of order (title auto-fills), expense detail, whether VAT was paid	§6.2 order_expenses; mandatory fiscal-invoice attachment option (§16 item 5)
12	General expenses table (non-order) with comment and VAT-paid flag	§6.3 general_expenses

#	Original requirement (summarised)	Where covered in this design
13	Tax summary dashboard: system-computed tax vs ZIMRA-assessed input, discrepancy, ZIMRA interest per current regulation; input VAT deducted from output VAT	§6.3 zimra_assessments; rules A.6; input-VAT deduction A.5 step 2; ZIMRA rate as effective-dated config with practitioner-verification note (§18.3 closing note)
14	Performance dashboard: all expenses vs contracts and orders; how much debt funded the business; pending expenses per period	Performance report preserved (§15.2); rules A.7; time-windowed successor metrics in the Command Centre (§14)
15	Financial summary dashboard with colour codes	Financial Summary preserved with classic HEALTHY/WATCH/ACT rules (A.8) alongside the new danger engine (§14.2)
16	Sample data to start; professional, industry-standard output	Replaced by scripted migration of the real workbook data with \$0.01 parity sign-off (§16 item 11, A.10)
17	A manual for the spreadsheet, included in the document	Gap closed in B.2(b): end-user documentation and training are now formal deliverables
18	"Check omissions and logical/operational oversights"	Fifteen-item oversight review in §16; the IMPROVE flags throughout Appendix A

B.2 Gaps found in this cross-check, now closed

(a) Partial servicing of orders

The original specification asked to see orders *partially serviced*; the workbook simplified Serviced to a yes/no tick, so partial delivery is invisible today. The system closes this: an order may optionally carry **service milestones** (description, percentage or value, date completed). Serviced % = completed milestone value ÷ order value. The health state machine (A.2) gains one state between Open and Awaiting Payment: **PARTIALLY SERVICED** (any milestone complete but not all), and dashboards show a serviced-percentage column. Orders without milestones behave exactly as today — the binary tick remains the default so migration parity (A.10) is unaffected.

(b) End-user documentation and training

Deliverables added to the delivery plan (§18.2): a role-based user manual (site clerk, custodian, finance, approver, director, admin) maintained as in-app help pages rather than a static document; short screen-recorded walkthroughs per module; and a train-the-trainer session for Finance at each phase's UAT. The admin panel links every screen to its help page so the manual can never drift far from the software.